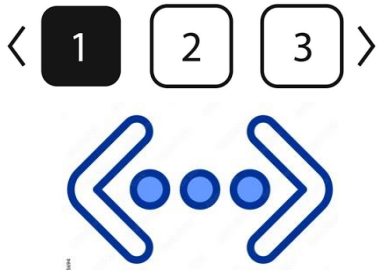
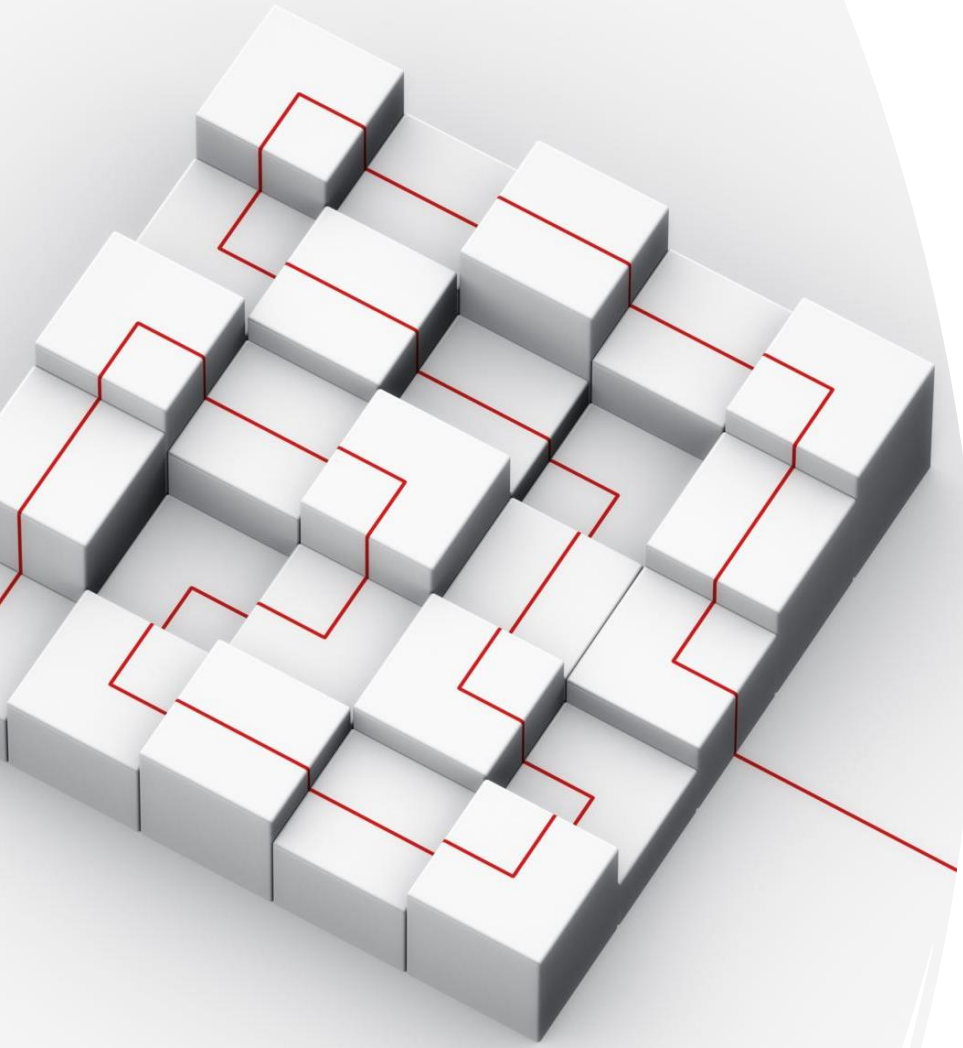
INT 161

Basic Backend Development

REST API Response Pagination and Data Transfer Object





Unit Objectives

- Explain why REST APIs use pagination for large collections of data.
- Describe the relationship between page, limit, and offset.
- Write MySQL queries with LIMIT, OFFSET, ORDER BY, and COUNT(*).
- Build a paginated REST API endpoint using Node.js and Express.js.
- Validate pagination query parameters and define safe limits.
- Design a consistent JSON response with pagination metadata.
- Explain the purpose of a Data Transfer Object (DTO).
- Distinguish between a database entity, a request DTO, and a response DTO.
- Design DTOs that avoid exposing sensitive or unnecessary database fields.
- Apply pagination and DTO concepts together to create a clear and maintainable API contract.

Why Pagination Matters

- Returning every record creates large responses.
- Large responses increase database work, memory use, and network transfer.
- Pagination gives clients predictable response sizes.

Pagination Parameters

- GET /api/products?page=2&limit=20
  - page: requested page number
  - limit: number of records per page
  - offset: number of records to skip (calculate)
- The Offset Formula:

```
OFFSET = (page - 1) x limit
```

```
const page = query.page || 1
const limit = query.limit || 20
const offset = (page - 1) * limit
```

page	limit	offset	take
1	10	0	1-10
2	10	10	11-20
3	10	20	21-30
9	10	80	81-90

Pagination Flow

→ Client request

→ Express route validates page and limit (or using middleware)

→ API calculates offset

→ MySQL retrieves records with LIMIT and OFFSET

→ API returns items and pagination metadata

MySQL Query

```
const TABLE_NAME = "products”
const ENTITY_NAME = "Product”

const COUNT_ALL = `SELECT COUNT(*) as total FROM ${TABLE_NAME}`
const FIND_ALL_SQL = `SELECT * FROM ${TABLE_NAME}`

async function count() {
  const [[{ total }]] = await db.query(COUNT_ALL);
  return total;
}
export async function findAll(pagination) {
  if (!pagination) {
     const [rows] = await db.query(FIND_ALL_SQL)
     return rows
  }
  const SQL = FIND_ALL_SQL + ' LIMIT ? OFFSET ?’
  const {limit, offset} = pagination
  const [rows] = await db.query(SQL, [limit, offset])
  rows.totalItems = await count()
  return rows
}
```

totalItems helps to calculate the total number of pages and pagination info.

JSON Response Structure


```
{
  "success": true,
  "data": [],
  "pagination": {
    "page": 2,
    "limit": 20,
    "totalItems": 138,
    "totalPages": 7,
    "hasNextPage": true,
    "hasPreviousPage": false
  }
}
```

Pagination Best Practices

- Validate that page is at least 1.
- Set a maximum limit, such as 100.
- Use SQL prepared statements.
- Create table indexes for filtering and sorting columns.
- Consider cursor pagination mechanism when users access very deep pages.

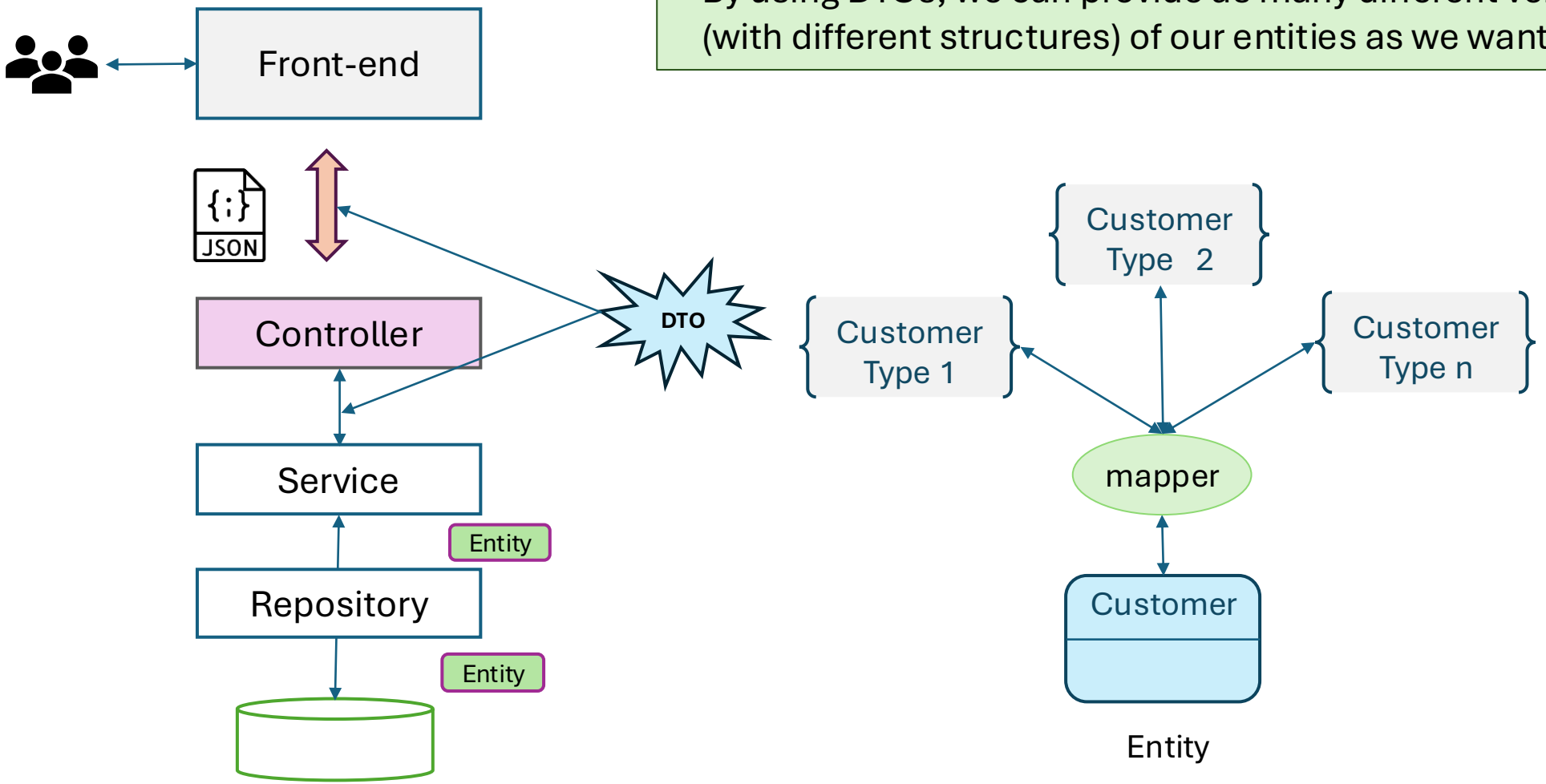
```
const query = req.query || {};
const pageSize = query.limit || 20
Math.max(parseInt(page), 1)
Math.min(parseInt(pageSize), 100)
const offset = (page - 1) * pageSize
```

DTO: Data Transfer Object

- A DTO is an object that defines data exchanged between API layers.
- It controls the shape of request and response data.
- A DTO separates external API contracts from database models.
- Why Use DTOs
  - Prevents accidental exposure of sensitive fields.
  - Makes API responses consistent.
  - Simplifies validation and documentation.
  - Reduces coupling between the database and API clients.

DTO: Data Transfer Object

- The data is mapped from the domain models to the DTOs.
- By using DTOs, we can provide as many different versions (with different structures) of our entities as we want.



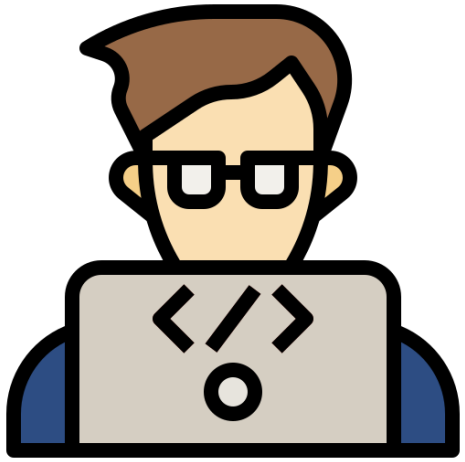
Entity vs DTO

- Database entity:
  - id, name, email, password_hash, role, created_at
- User response DTO:
  - id, name, email, role
- Entities represent stored data.
- DTOs represent data sent or received by the API.
- Never return fields such as password_hash.

Summary

- Pagination improves performance and usability for large lists.
- MySQL pagination commonly uses LIMIT and OFFSET.
- DTOs create clear, safe, and consistent API contracts.
- Keep database entities separate from API request and response objects.

Practices



Pagination Middleware

```
//src/middlewares/pagination-middleware.js
export const queryParser = () => {
    return (req, res, next) => {
        const query = req.query || {};
        if (!query || query.page == undefined || query.limit == undefined) {
            req.pagination = null
            return next();
        }
        let page = query.page || 1 //if empty, null set to 1
        const pageSize = query.limit || 20
        // console.log('page: ', page, 'size: ', pageSize)
        page = Math.max(parseInt(page), 1) //if page is negative
        const limit = Math.min(parseInt(pageSize), 100) //if page size > 100

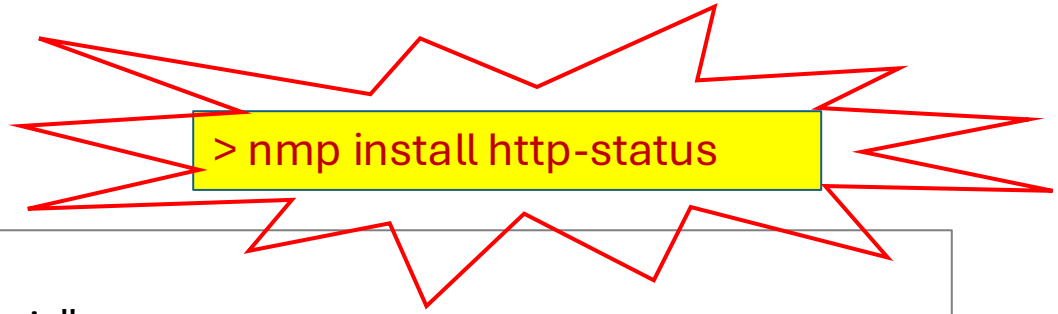
        const offset = (page - 1) * pageSize
        req.pagination = {page, limit, offset}
        next();
    }
}
```

Use Pagination Middleware with app.js

```
import {queryParser} from "./middlewares/pagination-middleware.js";
const ID = '6852000A'
const app express()
app.use(express.json())
app.use(queryParser()) //Must use before router

// Test queryParser
app.get('/', (req, res) => {
    console.log(req.pagination)
    res.json(req.pagination)
})
 :
 :
```

Product Repository (1)




```
import db from '../configs/db-config.js'
import {AppError} from "../middlewares/error-handler.js";
import httpStatus from "http-status";

const TABLE_NAME = "products"
const ENTITY_NAME = "Product"

const COUNT_ALL = `SELECT COUNT(*) as total FROM ${TABLE_NAME}`
const FIND_ALL_SQL = `SELECT * FROM ${TABLE_NAME}`
const FIND_ONE_SQL = `SELECT * FROM ${TABLE_NAME} WHERE productCode = ?`
const UPDATE_SQL = `UPDATE ${TABLE_NAME} SET ? WHERE productCode = ?`
const REMOVE_SQL = `DELETE FROM ${TABLE_NAME} WHERE productCode = ?`
const CREATE_SQL = `INSERT INTO ${TABLE_NAME} SET ?`

async function count() {
    const [[{ total }]] = await db.query(COUNT_ALL);
    return total;
}
```

Product Repository (2)

```
export async function findAll(pagination) {
    if (!pagination) {
        const [rows] = await db.query(FIND_ALL_SQL)
        return rows
    }
    const SQL = FIND_ALL_SQL + ' LIMIT ? OFFSET ?'
    const {limit, offset} = pagination
    const [rows] = await db.query(SQL, [limit, offset])
    rows.totalItems = await count()
    return rows
}
```

```
export async function findOne(id) {
    const [row] = await db.query(FIND_ONE_SQL, [id])
    if (row.length === 0) {
        throw new AppError(
            `${ENTITY_NAME} with code '${id}' was not found`,
            httpStatus.NOT_FOUND
        )
    }
    return row[0]
}
```

Product Mapper (1)

```
// src/mappers/product-mapper.js
export default class ProductMapper {
  // แปลงเป็น Less Data DTO (Summary)
   static toSummaryDTO(entity) {
      return {
         id: entity.productCode,
         name: entity.productName,
         scale: entity.productScale,
         retailPrice: entity.MSRP,
      };
   }
```

```
  // แปลงเป็น Full Data DTO
  static toDetailDTO(entity) {
     const {MSRP,buyPrice, ...data} = entity
     data.retailPrice = MSRP
     data.price = buyPrice
     return data
  }
```

Product Mapper (2)

```
static maps(entities, viewType) {
    const {totalItems} = entities
    let entitiesMap = {}
    if(viewType == 'full') {
        entitiesMap = entities.map(item => this.toDetailDTO(item))
    } else {
        entitiesMap = entities.map(item => this.toSummaryDTO(item))
    }
    if(totalItems) {
        entitiesMap.totalItems = totalItems
    }
    return entitiesMap
}
```

```
static map(entity, viewType) {
    if(viewType == 'summary') {
        return this.toSummaryDTO(entity)
    } else {
        return this.toDetailDTO(entity)
    }
}
}
```

Product Service

```
// src/services/product-service.js

import * as repo from "../repositories/product-repo.js"
import mapper from "../mappers/product-mapper.js"

export async function getObject(id,viewType) {
    return mapper.map(await repo.findOne(id));
}

export async function getAllObjects(pagination, viewType) {
    const objects = await repo.findAll(pagination)
    const objectsDto = mapper.maps(objects,viewType)
    console.log(objectsDto)
    return objectsDto
}
```

Api Response (1)




```
// src/mappers/api-response.js

import httpStatus from "http-status";

export async function apiResponse(res, data, pagination = null, status = httpStatus.OK) {
    return await (pagination ?
            paginate(res, data, pagination, status.valueOf()) :
            success(res, data, status.valueOf())
    )
}

// ส่ง Success Response ทั่วไป
async function success(res, data, statusCode) {
    return res.status(statusCode).json({
        success: true,
        data: data,
        timestamp: new Date().toISOString(),
    });
}
```

Api Response (2)

```
// ส่ง Success Response สำหรับข้อมูลแบบ Paginated
async function paginate(res, data, pagination, statusCode) {
    const {limit, page} = pagination
    const total = data.totalItems;
    return res.status(statusCode).json({
        success: true,
        data,
        pagination: {
            total: total,
            page: page,
            limit: limit,
            totalPages: Math.ceil(total / limit),
            hasNextPage: page * limit < total,
            hasPrevPage: page > 1
        },
        timestamp: new Date().toISOString()
    });
}
```

Product Router

```
// src/routes/product-route.js
import * as service from "../services/product-service.js"
import express from "express"
import {apiResponse} from "../mappers/api-response.js";

const router = express.Router()

router.get("/", async (req, res) => {
    const pagination = req.pagination
    const viewType = req.query.view || "summary"
    const objects = await service.getAllObjects(pagination, viewType)
    apiResponse(res, objects, pagination)
})
```

```
router.get("/:id", async (req, res) => {
    const viewType = req.query.view || "full"
    const id = req.params.id
    const object = await service.getObject(id,viewType)

    apiResponse(res, object)
})
```